ISSN 1009-3044
Computer Knowledge and Technology 电脑知识与技术
Vol.21, No.18, June 2025

E-mail：eduf@dnzs.net.cn
http://www.dnzs.net.cn
Tel：+86-551-65690963 65690964

AI+Coding：探索智能编程的实现路径与机遇

蒋晓薇

(西安外事学院 工学院，陕西 西安 710777)

摘要：随着人工智能技术的快速发展，人工智能与编程的融合已成为软件开发领域的必然趋势。文章详细分析了人工智能加编程的基本概念与典型应用场景，基于AI大模型工具Trae对实现智能编程的过程进行了案例演示，最后对人工智能时代背景下程序员的职业前景与转型方向进行了思考。文章为行业实践提供了可复用的技术路径，同时为程序员的职业发展规划提供了重要参考。

关键词：人工智能；智能编程；AI大模型；程序员

中图分类号：TP18 **文献标识码：**A

文章编号：1009-3044(2025)18-0014-04

DOI:10.14004/j.cnki.ckt.2025.0886

开放科学(资源服务) 标识码(OSID)：

0 引言

在第四次科技革命的洪流中，人工智能(AI) 技术迅速融入人们生产生活的各个方面，成为推动行业变革的关键驱动力。尤其在软件开发领域，AI与编程(Coding) 的结合正在重塑编程范式，为编程效率和质量的提升提供了新的可能，也为解决传统编程模式面临的诸多挑战提供了新的思路和方法。

传统的编程模式因高代码重复率、开发周期长以及维护成本高而面临严峻挑战，亟须借助AI技术提质增效。通过AI与Coding的深度融合，程序员可以将精力从烦琐的代码语法中解放出来，专注于系统需求分析、架构设计和复杂逻辑的实现，从而提高软件开发的整体效率和质量。

本文将深入探讨AI+Coding的基本概念、应用场景、实现路径以及对程序员职业前景的影响和思考，旨在为软件开发人员和相关从业者提供一个全面的视角，以期更好地应对这一新兴趋势的挑战。

1 人工智能与编程融合的必要性

1.1 AI技术的迅猛发展

AI技术的蓬勃发展已经成为推动各行各业变革的中坚力量。开源模型DeepSeek的出现更是为产业带来新的活力，大大降低了技术门槛。AI+教育、AI+医疗、AI+自媒体等领域，AI的应用场景不断拓展，为各行各业的生产力带来爆发式增长，促进了产业生态的多元化发展。

1.2 软件开发领域的现状与挑战

在软件开发领域，目前面临的核心挑战主要是工作效率与质量控制之间的平衡。程序员在工作中需要投入大量精力应对重复性劳动，例如，易长安等[1]研究发现，重复代码在软件项目中十分普遍，它是导致软件维护成本高的主要原因之一。大量的重复代码会导致运行时间急剧增加，吴冲等[2]表明，对臃肿代码的维护时间占据软件生命周期的50%甚至更高。另外，系统调试与异常处理也在持续消耗开发团队的研发成本，导致项目周期不断延长。这种效率损耗会随着项目规模的增加呈现指数级增长，最终形成陷入软件持续优化的瓶颈。

1.3 AI与编程融合的必然性

综合来看，AI技术的发展与软件开发面临的挑战形成了双轮驱动：一方面，AI技术需要软件开发作为实现载体；另一方面，软件开发迫切需要借助AI技术打破效率瓶颈。因此，AI与Coding的融合迫在眉睫，利用AI技术对代码进行实时分析和优化，检测潜在的风险和错误，从而提高编码质量，降低维护成本。

综上所述，无论从AI迅猛的发展趋势这一外部拉力推动，还是软件开发领域自身提质增效的内需驱动，都亟须软件开发领域将AI与Coding相融合，实现生产力的跃迁。

2 AI+Coding的基本概念与典型应用场景

2.1 AI+Coding的基本概念

AI+Coding是指利用人工智能技术辅助软件开发过程中的编码环节，借助AI大模型理解编程需求，自动生成或辅助生成代码，同时对代码的质量进行评估和优化[3]。AI大模型工具通过对大量代码数据的学习和分析，能够理解代码中的逻辑，然后根据输入的意图描述自动补全或生成可执行的代码[4]，构建“人机协同”的智能编程范式，实现编程过程的自动化和智能化。

收稿日期：2025-02-05
作者简介：蒋晓薇(1991—)，女，工程师，硕士，研究方向为软件测试、软件工程、大数据技术。

中国知网 https://www.cnki.net

2.2 AI+Coding的典型应用场景

智能编程的典型应用场景主要有代码自动生成、代码补全、错误检测与修复、代码解释、代码优化[4]。

1) 代码自动生成。代码自动生成是指AI工具根据输入的需求文档或自然语言描述，自动将其转化为可运行的代码。例如，在AI工具对话框中输入“请基于web技术栈帮我生成一个贪吃蛇游戏”，大模型会自动完成HTML、JavaScript和CSS文件的创建以及代码的编写，给用户呈现一个可以执行的贪吃蛇小游戏。这极大简化了软件开发过程，不仅提高了代码编写效率，还能减少人为错误，提高开发质量。但是，在与AI工具对话时，明确开发的技术栈是提高AI工具开发效率的必要手段。

2) 代码补全。代码补全是指在编码的过程中，AI工具根据已输入的代码和上下文信息，实时预测并自动补全后续代码，生成完整的代码块，从而极大提高开发速度。例如，当程序员输入函数名的前几个字母时，AI工具会自动补全完整的函数名、参数列表及剩余的代码逻辑，或者根据注释信息自动补全相应的代码。这可以大幅减少程序员在编写代码时的重复性劳动，降低因拼写错误导致的代码报错。然而，代码补全的准确性高度依赖于训练数据的数量和质量。如果训练数据不完整或存在偏差，AI工具可能会生成不准确的代码片段。

3) 错误检测与修复。错误检测与修复是指通过AI工具实时检测代码中的拼写和语法等错误，识别潜在的报错，并给出修复建议，从而修正代码中的错误。例如，程序员在创建一个函数时少写了括号，AI工具可以自动检测并修复该错误。或者在循环语句中使用了错误的比较条件，AI工具能够检测并提供相应的修复建议。这可以减少代码调试的时间，提升开发体验。但对于复杂项目，AI可能无法准确识别逻辑上的错误。

4) 代码解释。代码解释是指利用AI技术对代码的框架、逻辑等进行分析，拆解代码的功能结构，将代码转化为易于理解的自然语言叙述。例如，对于一段判断闰年的Python代码，AI工具不仅可以解释代码的功能，还可以梳理清楚代码的输入和输出。这有助于程序员在阅读和维护他人代码时，快速理清代码的功能和作用。但对于多层嵌套函数、递归调用或动态数据结构等较为复杂的代码逻辑，AI可能无法提供准确的自然语言解释。

5) 代码优化。代码优化是指通过AI技术分析代码的质量和性能瓶颈，提出优化改进建议或自动执行优化操作以改善代码的性能。例如，AI工具可以检测到函数循环体中的低效操作，并提供优化方案。AI工具还可执行死代码消除、内存优化等操作，从而有效提升软件的执行效率、降低资源损耗并增强代码的可维护性。但由于上下文的限制，对于大型项目，特别是复杂场景的优化能力有限，可能导致修改建议不准确或不符合预期。

3 实现智能编程的路径与案例演示

3.1 实现智能编程的路径

当前实现AI+Coding智能编程的路径主要有两种。第一类是在常见的集成开发环境(IDE)中添加AI编程助手插件，即在常见的IDE工具(*VS Code*、*IntelliJ IDEA*等)中下载安装AI编程助手插件。主流的AI编程助手有国内的DeepSeek推理模型、豆包的MarsCode模型、阿里的通义灵码、腾讯云的AI代码助手，这些均可免费使用。国际上的编码模型有*Claude 3.7 Sonnet*、*Github Copilot*、*Gemini 2.0*等。国内和国际厂商的AI编程助手模型具体如表1中的“IDE工具+AI编程助手”一列所示。表1中“AI IDE”这一列是实现智能编程的第二类路径，即将AI模型高度集成在IDE环境之中，如国内字节跳动推出的*Trae*工具，国际厂商推出的Cursor工具。相比于AI编程助手，AI IDE拥有更强的上下文感知能力和全场景的处理能力，可以带来更加流畅、准确和优质的开发体验。

表1　智能编程主流模型与工具

IDE工具+AI编程助手		AI IDE
国内	DeepSeek	Trae
	豆包*MarsCode*	
	通义灵码	
	腾讯云AI代码助手	
国际	Claude 3.7 Sonnet	Cursor
	Github Copilot	
	Gemini2.0	

*Trae*和*Cursor*是两款备受关注的AI IDE工具，其中Trae是字节跳动推出的国内首个AI IDE，于2025年3月3日正式发布，国内用户可以免费下载使用，其国际版本较国内提前2个月发布。另外，Cursor则是在国际市场成熟度较高的AI IDE工具。两者均基于*VS Code*生态开发，但在功能覆盖和模型支持上存在差异。国内*Trae*工具与国际*Cursor*工具在核心功能方面的对比如表2所示。

表2　国内*Trae*工具 vs. 国际*Cursor*工具

核心功能	Trae	Cursor
模型覆盖	*Doubao-1.5-pro DeepSeek R1/ V3*	Claude 3.7 Sonnet, GPT-4o, DeepSeek R1/ V3, Gemini 2.0
多模态支持	文本交互	文本、图片交互
跨平台支持	*MacOS*, Windows	*MacOS*, Windows, Linux
交互模式	Builder窗口、Chat窗口、内联窗口	Composer窗口、Chat窗口、内联窗口、终端窗口
上下文管理	Code, File, Folder	Code, File, Folder, Codebase, git等
代码补全	Tab键采纳	Tab键采纳

两个工具的差异主要体现在模型覆盖、多模态支持、跨平台支持和上下文管理等方面。

模型覆盖：*Cursor*支持的模型更广泛(如*GPT-4o*、

中国知网 https://www.cnki.net

Gemini 2.0)，而 *Trae* 国内版侧重国产模型。

多模态支持：*Trae* 国内版仅支持文本交互，Cursor 支持文本和图片交互。

跨平台支持：*Cursor* 额外支持 Linux 系统。

上下文管理：*Cursor* 的代码库级管理和 Git 集成更适用于复杂项目。

3.2 基于工具 *Trae* 的智能编程案例实施

1) Trae 的安装与使用

登录官网下载安装 *Trae* 工具，并在 Builder 窗口中输入需求：请基于 web 技术栈生成一个贪吃蛇小游戏。*Trae* 接收需求并进行思考，规划完成本次需求的核心逻辑步骤，*Trae* 接收需求与思考过程如图 1 所示。

图1　*Trae* 接收需求与思考过程图

等待约6分钟，*Trae* 完成贪吃蛇小游戏核心文件的创建和代码编写，并在窗口输出本次创建的核心文件及实现的主要功能。

2) 需求变更与代码修改

笔者打开创建好的贪吃蛇小游戏进行测试，测试过程中发现蛇移动速度过快，游戏体验不佳。返回工具并提出调慢蛇移动速度的需求，同时将“速度”二字误写为“方向”，以测试工具是否能够理解并识别真实需求。工具接收需求变更与思考过程如图 2 所示，从图中可以看出 *Trae* 能够正确识别调整“速度”的需求，并将变量值 gameSpeed 由 100 ms 修改为 150 ms。

3) 多需求处理与优化

Trae 完成修改后，笔者打开游戏继续进行测试，发现蛇移动速度仍然过快，且游戏界面没有开始游戏按钮，打开页面后立即开始游戏，游戏体验不佳。于是返回工具，继续提出两个修改需求：“添加游戏开始按钮，蛇的移动速度调到 200 ms”，测试大模型是否能够准确识别多个需求并完成修改。

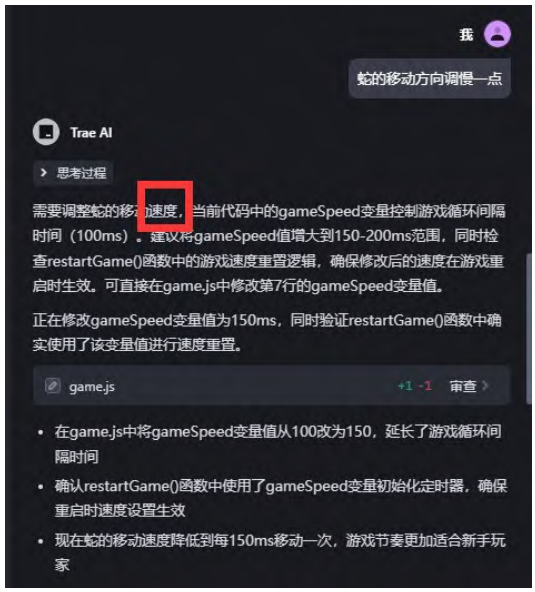

图2　*Trae* 接收需求变更与思考过程图

Trae 接收这两个需求并进行思考的过程如图 3 所示，从图中可以看出 *Trae* 能够正确识别出两个调整需求，并且智能地按照实现难易程度对两个需求的顺序进行了调整。优先处理蛇的移动速度这一需求，添加游戏开始按钮的需求放在第二步进行处理。因为调整蛇的移动速度只需修改变量 gameSpeed 的值，实现过程较为简单；而添加开始按钮则需要修改 3 个文件的处理逻辑，实现过程更加复杂。

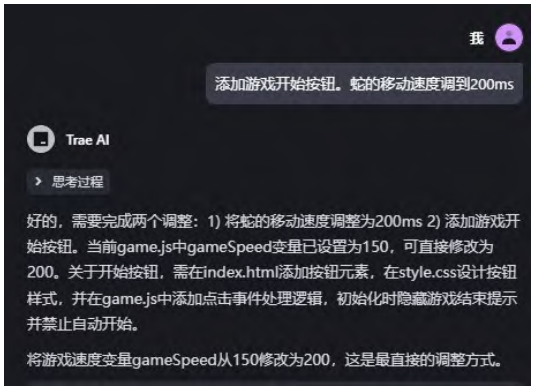

图3　*Trae* 接收两个需求与思考过程图

从对这两个需求的识别与排序处理这一角度来看，*Trae* 在代码自动生成方面的表现较为优异。最终，*Trae* 根据用户输入的需求实现的贪吃蛇小游戏界面如图 4 所示，界面简洁清晰，按照用户需求实现了“开始游戏”按钮的添加，试玩时贪吃蛇移动速度适中，整体表现良好。

4 AI 时代程序员面临的机遇与挑战

AI 大模型在代码自动生成、代码补全、错误检测与修复等方面表现出色，能够高效地完成诸如贪吃蛇小游戏等简单需求的代码实现。但这并不代表程序员在 AI 时代将被完全取代或面临失业的困境。当前 AI 技术的发展仍然存在许多局限性，这些局限性在一

中国知网 https://www.cnki.net

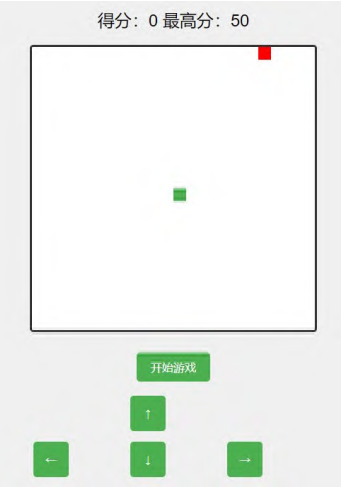

图4　贪吃蛇小游戏界面

定程度上决定了未来程序员在软件开发领域中仍会扮演不可或缺的角色。

4.1 开放性问题表现一般：复杂项目的挑战

AI在处理具有统一标准和客观答案的任务时表现突出，例如在进行语言翻译或者数学运算时，能够快速准确地输出结果。但是在处理没有固定答案的开放性问题或复杂项目时表现欠佳。这类问题需要结合多方因素进行综合考量，AI在理解和生成创新性解决方案方面的能力有限，很难像人类一样灵活应对复杂多变的项目状况。随着项目规模和复杂性的增大，AI出错的概率会明显提高。由于AI很难梳理清楚大型项目中各模块的交互逻辑，因此需要人类程序员与客户进行反复沟通来确认需求的边界和规则，并对需求进行拆分，尽量让AI一次处理一个小模块的小需求。因此，在开放性需求和复杂项目中，程序员依然需要主导需求的分析整合以及架构设计。

4.2 上下文窗口的限制：记忆与理解的瓶颈

上下文窗口是指AI模型在生成或理解语言时可以接收的文本量，或模型可以处理的token数。它主要影响的是模型在处理信息时可以利用的前后文信息量或生成回复时的token数量。吴国栋等[5]表明，目前的大语言模型存在上下文长度有限的问题。即，大语言模型存在上下文窗口或者token的限制，一旦用户的输入超过这个上限，系统只会截取其中一部分信息进行理解和回答，这会导致AI出现信息丢失或者遗忘的现象。在实际的项目开发过程中，程序员需要对成千上万行代码、众多的功能模块和复杂的业务逻辑进行处理，上下文窗口的限制使得AI很难保持始终如一的理解与记忆。当前AI难以在大规模的应用场景中完全替代人类程序员。在上下文记忆方面，程序员应将长篇项目拆解后再调用AI进行多次迭代，在调用AI工具时采取相应措施，如分段处理代码、手动整合信息等，以弥补AI上下文窗口限制导致的记忆与理解的瓶颈这一不足。

4.3 幻觉问题：可靠性与安全性隐患

AI的幻觉问题是指人工智能系统会生成一些看似合理、通顺但实际错误或虚构甚至完全不合理的信息。正如何静等[6]提到，这种幻觉现象一般出现在大语言模型中，会影响模型输出的可靠性和安全性。在实际的项目开发过程中，需要开发、测试、运维等各个部门之间的紧密协作，目前AI的幻觉问题导致其可能会生成存在安全隐患的代码，可靠性在长时间内无法达到人类程序员的标准。针对AI的幻觉问题，程序员需要对其生成的代码进行严格审查，特别是涉及安全相关的代码、业务逻辑的实现等方面，要确保代码的安全合规。

综上所述，AI处理复杂项目的能力有限，很难准确拆分复杂需求，应对各种突发状况；上下文窗口的限制使得AI在理解和记忆长篇代码时存在瓶颈；幻觉问题更是让AI生成的代码在可靠性和安全性上存在隐患。这些局限性都表明，人类程序员不仅不会被取代，反而需要不断提升自身能力，与AI紧密协作，共同推动软件开发领域提质增效诉求的落地。AI技术的发展为程序员带来了挑战，同时，技术的局限性也为程序员创造了新的机遇。

5 总结

AI与Coding的深度融合是软件开发领域的一次深刻变革，开启了智能编程的新范式。本文详细探讨了AI+Coding的基本概念、典型应用场景以及实现路径，并通过*Trae*工具展示其在代码生成、调试优化等方面的应用效果。尽管AI技术在代码自动生成、代码补全、错误检测与修复等方面表现出色，但在处理复杂项目和开放性问题时仍显不足。

这些发现为软件开发人员提供了实用的参考，帮助他们更好地理解和应用AI技术，从而在AI时代保持竞争力。未来，智能编程的研究方向应聚焦于如何在大规模企业项目中克服当前AI技术的局限性，这包括但不限于探索AI与人类程序员协同的最优实践，以及开发能够处理复杂逻辑和开放性问题的AI模型。

参考文献：

[1] 易长安.重复代码检测技术的现究现状[J].电脑知识与技术,2009(8):6229-6230,6237.
[2] 吴冲,吴海涛.重复代码的重构[J].电脑与信息技术,2015,23(2):38-42.
[3] 邢万勇,廖伟军,杨智帆.基于AI编程助手的软件研发技术效能提升实践[J].数字技术与应用,2025,43(1):28-30.
[4] 崔娜,刘雯,陈岩.AIGC在编程领域的应用与探究[J].智慧中国,2024(7):48-49.
[5] 吴国栋,秦辉,胡全兴,等.大语言模型及其个性化推荐研究[J].智能系统学报,2024,19(6):1351-1365.
[6] 何静,沈阳,谢润锋.大语言模型幻觉现象的分类识别与优化研究[J].计算机科学与探索,2025,19(5):1295-1301.

【通联编辑：唐一东】

中国知网 https://www.cnki.net